RAG Assignment: Baseline Retrieval vs. Cross-Encoder Re-Ranking

Sankalp Chaturvedi
2025201083

GitHub: https://github.com/Sankalp0109/RAG_ASSIGNMENT_LMA

1 Setup & Environment

Dataset. The 2024 FIA Formula One Sporting Regulations (Issue 6, 106 pages), a regulatory / rulebook-style PDF. Most rules are followed by "unless", "except" or "may not" clauses, and the same words (*pit lane, penalty, tyres, safety car*) recur across many unrelated articles, which makes it a hard case for dense retrieval.

LLM & embeddings. Bi-encoder: `sentence-transformers/all-MiniLM-L6-v2` (384-d). Cross-encoder: `cross-encoder/ms-marco-MiniLM-L-6-v2`. Both run locally (PyTorch, Apple M4 / MPS). LLM: Gemini API, `gemini-3.6-flash`, temperature 0.

2 Two-Stage Retrieval Implementation

Chunk corpus. Text is extracted with PyMuPDF and split into one chunk per sub-article (e.g. 55.8). Long sub-articles are split at $\leq$900 characters with a 150-character overlap, giving 526 chunks. Both pipelines use exactly this chunk set.

Pipeline 1 (Baseline Retrieval). (a) All chunks are embedded with the bi-encoder and indexed in a FAISS inner-product index on L2-normalised vectors, so the score equals cosine similarity. (b) For each query the Top-10 chunks are retrieved. (c) The Top-3 of these are passed to the LLM to generate *Answer 1*.

Pipeline 2 (Advanced Re-Ranking). (a) The same Top-10 candidates are taken. (b) All 10 (query, chunk) pairs are scored by the cross-encoder. (c) The candidates are re-sorted by cross-encoder score and the Top-3 are kept. (d) These are passed to the LLM with the *identical prompt* to generate *Answer 2*. The prompt tells the model to answer only from the three passages, cite article numbers, and otherwise reply "The provided context does not answer this".

3 Failure Mode Analysis (Stress Testing)

I built a test suite of 52 questions. Each is labelled with the exact sentence in the PDF that answers it (the gold chunk must contain that sentence) and tagged as keyword trap (10), exception / corner case (13), negation / constraint (15), buried chunk (3), or a direct-lookup control (11). Below are the failure queries where the baseline stumbles and re-ranking restores relevance, grouped by the categories in the assignment.

3.1 Keyword Trap / Distractor

Q26: "Can children enter the pit lane?" "Children" never appears in the document; the rule says "no one under the age of 16". The bi-encoder fills its Top-3 with chunks that merely share the words *pit lane* (a car reversing in the pit lane, the pit exit being closed). The cross-encoder connects the synonym and promotes the rule from #5 to #2.

3.2 Exceptions & Corner Cases

Q10: "Must every driver use two different tyre compounds in the race?" and **Q50: "What happens if a driver cannot serve a penalty because he retired?"** The two-compound tyre rule sits inside the 5-page tyre article. That article yields about ten near-identical chunks with cosine scores between 0.57 and 0.68, so the bi-encoder orders them almost at random and the rule lands at #8. The cross-encoder separates them sharply: the gold chunk scores 7.1 against at most 3.6 for the rest, so it moves to #1. Q50 is a true "except when" clause (a penalty that cannot be served because the driver retired) hidden in the

middle of the long 54.3 penalty list. It moves from #6 to #1 with a cross-encoder score of $+5.5$, while every other candidate scores below -0.2.

	Chunk	Bi rank	Cosine	CE rank	CE score	Content
	30.3	1	0.683	2	3.62	Presenting tyres to the Technical Delegate
	30.5	2	0.680	5	2.19	"All tyres must be operated in accordance with …"
	30.5	3	0.653	4	2.26	Same-specification sets; 54.3d penalty
	30.3	4	0.642	6	0.32	Marking of the tyre sidewall
	30.5	5	0.607	7	0.16	Additional tyres made available by the FIA
	30.6	6	0.582	10	−5.37	Testing of tyres
	30.2	7	0.581	9	−2.29	Tyre allocation per Competition
⋆	**30.5m**	**8**	**0.580**	**1**	**7.08**	**"must use at least two different specifications"**
	30.5	9	0.575	8	−1.29	Wet-weather tyres when a race is resumed
	30.2	10	0.571	3	2.59	Allocation when extra sets are released

Table 1: Full Top-10 for Q10 ("Must every driver use two different tyre compounds in the race?"). Cosine scores are almost flat (0.57–0.68), so the bi-encoder cannot tell the tyre chunks apart. The cross-encoder gives the gold chunk ($\star$) a clear margin (7.08 vs. 3.62) and moves it #8 $\rightarrow$ #1.

3.3 Negation & Constraints

Q31: "When are no championship points awarded for a shortened race?" and **Q21: "Can mechanics work on the car while a driver serves a five second penalty?"** For "when are *no* points awarded", the bi-encoder's top hits are the *positive* points table (6.4) and the dead-heat rule (7.1). These are on-topic but have the wrong polarity, and Answer 1 says the context does not answer. For Q21 ("may not be worked on") the baseline Top-3 held the neighbouring stop-and-go clause. Answer 1 reaches the right yes/no answer but cites the wrong clause, and adds an engine-restart allowance that does not apply to a 5 s penalty. Re-ranking moves the gold chunk from #4 to #2 in both queries.

3.4 Other Document-Specific Failures

Q12: "What is the penalty for not using two different dry tyre specifications?", a vital chunk buried at ranks 4–10. This is the most harmful baseline case. The gold chunk sat at #9, and the baseline Top-3 contained a *different* tyre penalty (54.3d, for not changing tyres as instructed). Answer 1 presented that penalty as the answer: a confident, cited, but wrong rule rather than a refusal. After re-ranking (#9 $\rightarrow$ #1), Answer 2 gives the correct sanction: disqualification, or +30 s if the race cannot be restarted.

4 Analysis and Report

4.1 Architecture Comparison

A **bi-encoder** encodes the query and each chunk *independently* into one fixed vector ($\mathbf{q} = f(q)$, $\mathbf{d} = f(d)$) and scores them by cosine similarity $\cos(\mathbf{q}, \mathbf{d})$. Since $\mathbf{d}$ does not depend on the query, all chunks are embedded once, offline, and retrieval is just a nearest-neighbour search. The limitation is that each chunk is compressed into a single 384-dimensional vector before the question is known, and query and chunk only meet in one final dot product. No query token ever attends to a passage token. Fine distinctions such as "*no* points are awarded" vs. "points are awarded", or "children" vs. "under the age of 16", get averaged into a general topic vector.

A **cross-encoder** feeds the pair together, `[CLS] query [SEP] chunk [SEP]`, through the transformer and predicts a relevance score from `[CLS]`. Self-attention runs *jointly across the tokens of both texts* in every layer, so each query word can attend to each passage word. The model can check whether the passage actually answers the question (right condition, right polarity, right actor) instead of just sharing its topic. The cost is that the score depends on the query, so nothing can be precomputed: every (query, chunk) pair needs a full forward pass at query time.

4.2 Qualitative & Rank Shift Analysis

		Hit@3 (gold reaches the LLM)		MRR@10	
Query category	n	Baseline	Re-ranked	Baseline	Re-ranked
Negation / constraint	15	0.73	**0.87**	0.65	**0.75**
Exception / corner case	13	0.85	**0.92**	0.64	**0.83**
Keyword trap / distractor	10	0.60	0.60	0.50	**0.57**
Buried chunk (rank 4–10)	3	0.33	**0.67**	0.20	**0.73**
Control (direct lookup)	11	1.00	1.00	**0.95**	0.94
All	52	0.769	**0.846**	0.656	**0.774**

Table 2: Retrieval quality per failure category: how often the gold chunk reaches the LLM's Top-3 (Hit@3) and how high it is ranked (MRR@10). Hit@1 rises from 0.519 to 0.692. The gold chunk is inside the baseline Top-10 for 47/52 queries.

Re-ranking leaves direct lookups unchanged, where the bi-encoder is already near-perfect. It helps most where the answer depends on a condition or on polarity: negation (+14 points of Hit@3), exceptions (+7) and buried clauses. Table 3 lists every query whose gold chunk crossed the Top-3 boundary, i.e. every case where re-ranking changed what the LLM was given. Table 1 shows one full Top-10 in detail, and Table 4 quotes the two answers for three representative cases.

ID	Query (gold article)	Rank shift	Answer 1 (baseline) vs. Answer 2 (re-ranked)
Q12	Penalty for not using two dry tyre specs? (30.5m)	#9 → #1 ↑	A1 *misstates the rule*, gives an unrelated 54.3d penalty; A2 correct (DSQ / +30 s)
Q10	Must every driver use two different compounds? (30.5m)	#8 → #1 ↑	A1 "context does not answer"; A2 states the rule but omits the intermediates exception
Q50	Penalty that cannot be served due to retirement? (54.3)	#6 → #1 ↑	A1 refuses; A2 correct (grid penalty at next race)
Q26	Can children enter the pit lane? (26.13)	#5 → #2 ↑	A1 refuses; A2 gives the under-16 rule and time windows
Q21	Can mechanics work on the car during a 5 s penalty? (54.4c)	#4 → #2 ↑	A1 right yes/no but cites the wrong clause (54.4d); A2 cites 54.4c
Q31	When are no points awarded for a shortened race? (6.5a–b)	#4 → #2 ↑	A1 refuses; A2 lists both no-points conditions
Q03	Top up fuel when the car pits for tyres? (36.2)	#3 → #5 ↓	A1 *correct*; A2 refuses (regression)
Q47	How often must a rookie run in practice? (32.4c)	#3 → #5 ↓	gold leaves the context (not sent to the LLM)

Table 3: All queries whose gold chunk crossed the Top-3 boundary. Full Top-10 tables with bi-encoder and cross-encoder scores are in the notebook (Section 7).

Did the baseline hallucinate, omit a clause, or misstate a rule? In all six rescued queries the critical chunk was outside the baseline Top-3, and Answer 1 failed in one of three ways. *Omission* (Q10, Q26, Q31, Q50): the model said the context does not answer the question. *Misstated rule* (Q12): it presented an unrelated penalty as the answer. *Wrong clause* (Q21): the right yes/no answer, justified with a neighbouring clause plus a condition that does not apply. No answer invented facts absent from its passages, because the prompt restricts the model to the context. The failures were therefore missing or misattributed rules, not free hallucination. With the critical chunk in its Top-3, Answer 2 was correct in all six; Q10 still lacks the exception, for the chunking reason below.

Where re-ranking did not help. (i) In Q03 the question mixes two topics (fuel *and* tyres). The cross-encoder followed *tyres* and pushed the refuelling chunk out of the Top-3, turning a correct Answer 1 into a refusal. All of its scores for this query were strongly negative (-5.6 to -10.7), so it was not confident about any candidate. (ii) For 5 queries the gold chunk was not in the Top-10 at all, so a re-ranker over the

Query	Answer 1 (baseline Top-3)	Answer 2 (re-ranked Top-3)
Q12 #9→#1	"According to Article 30.5, a penalty under Article 54.3d) will be imposed on any driver who does not change tyres as specified." *Wrong rule: misstated penalty.*	"Failure to use at least two different specifications of dry-weather tyres during a race will result in the disqualification of the driver from the race results (Article 30.5). However, if the race is suspended and cannot be re-started, … thirty (30) seconds will instead be added …" *Correct.*
Q31 #4→#2	"The provided context does not answer this." *Omission: gold not in context.*	"… no points are awarded if the leader has completed less than two laps (Article 6.5a). Additionally, no points are awarded unless a minimum of two laps have been completed by the leader without a Safety Car and/or VSC intervention (Article 6.5b)." *Correct.*
Q03 #3→#5	"According to Article 36.1, refuelling is only permitted in the Competitors' designated garages. Furthermore, under Article 36.2, fuel may not be added to a car after it has left the pit lane … until the end-of-session signal has been shown." *Correct.*	"The provided context does not answer this." *Regression: re-ranking removed the gold chunk.*

Table 4: Verbatim LLM answers (identical prompt, only the Top-3 chunks differ; "…" marks trimmed text). All 14 answers are in the notebook.

Top-10 cannot recover it. (iii) In Q10 the chunk boundary splits clause 30.5m, so the words "Unless he has used intermediate or wet-weather tyres" are in the previous chunk. Answer 2 therefore states the rule without its exception: re-ranking fixes ordering, not chunking.

4.3 Latency Profiling

Retrieval stages were timed over all 52 queries $\times$ 20 repeats (1,040 runs, after warm-up). LLM time is the wall-clock time of the 14 Gemini calls in the failure-query analysis.

Stage	Mean (ms)	p50 (ms)	p95 (ms)
Query embedding (bi-encoder)	5.32	5.30	5.66
FAISS Top-10 search (526 vectors)	0.02	0.02	0.03
Cross-encoder re-rank (10 pairs)	44.70	45.07	50.13
Pipeline 1 retrieval (baseline)	**5.34**	5.32	5.69
Pipeline 2 retrieval (re-ranking)	**50.04**	50.47	55.55
LLM generation (Gemini API, per call)	6,526	5,275	12,231
Pipeline 1 end-to-end	6,531	5,280	12,236
Pipeline 2 end-to-end	6,576	5,325	12,281

Table 5: Latency per query (Apple M4, MPS). Embedding all 526 chunks and building the index is a one-off 2.0 s.

Re-ranking makes retrieval about $9\times$ slower (5 ms $\rightarrow$ 50 ms), but in absolute terms it adds only 45 ms. That is about 0.7% of end-to-end latency, because the LLM call (5–12 s) dominates everything. Ten cross-encoder passes (about 4.5 ms per pair) are a small price for the gains in Table 2.

4.4 Scaling Question

Chunks	Bi-encoder (flat search)	Two-stage (+ re-rank Top-10)	Cross-encoder on *all* chunks
526 (this corpus)	5.3 ms	49.6 ms	3.3 s
10,000	5.7 ms	50.0 ms	1.1 min
100,000	9.7 ms	53.9 ms	10.5 min
1,000,000	49.1 ms	93.4 ms	1.8 h
10,000,000	444 ms	488 ms	17.6 h

Table 6: Per-query cost. The first row is measured; larger rows extrapolate linearly from the measured costs (6.3 ms per cross-encoder pair, and flat-search time scaled with corpus size).

We do not run the cross-encoder over the whole collection because its cost grows linearly with the collection and cannot be moved offline. To test this, I scored all 526 chunks with the cross-encoder for a single query: it took 3.3 s, about 600× the bi-encoder retrieval. A cross-encoder score exists only for a specific (query, chunk) pair, so for *every* new question the full transformer must run once per chunk. With a bi-encoder, the chunk-side work (embedding) is done once, ahead of time. Each query then costs one short encoding plus a vector search, and an approximate-nearest-neighbour index can keep that search fast even for millions of vectors. Extrapolating, a cross-encoder over one million chunks would take about 1.8 h per question, compared with under 0.1 s for the two-stage pipeline (Table 6).

Scoring everything would also add little accuracy, because the cheap first stage already has high recall (the gold chunk was in the Top-10 for 47 of 52 queries). The sensible design is therefore a funnel. The fast, high-recall bi-encoder narrows the collection to a few candidates, and the slow, high-precision cross-encoder is spent only on ordering those candidates. That is exactly where it changed the answers in Table 3.

5 Conclusion

On a regulation-heavy document, the bi-encoder reliably finds the right *topic* but often misses the right *clause*, usually the one that carries an exception, a negation, or a synonym of the query's wording. Re-ranking its own Top-10 with a cross-encoder fixed many of these cases: Hit@3 rose from 0.769 to 0.846, MRR@10 from 0.656 to 0.774, and the gains came from negation, exception and buried-clause questions, not from simple lookups. In every query where re-ranking pulled the critical chunk into the Top-3, the answer changed from an omission or a misstated rule to a correct, cited answer. The extra cost was about 45 ms per query, under 1% of the end-to-end time.

Re-ranking is not a complete fix. It can only reorder what the first stage retrieved (5 of 52 gold chunks were never in the Top-10). It cannot repair a chunk boundary that separates a rule from its exception (Q10). On a mixed-topic question it demoted the right chunk and made the answer worse (Q03). The cross-encoder is also far too slow to score a whole collection (about 1.8 h per query at one million chunks). The two models therefore work best together: the bi-encoder for fast, high-recall candidate retrieval, and the cross-encoder for precise ordering of those few candidates before they reach the LLM.